INT3404 - IMAGE PROCESSING
Assignment 1
The Digital Camera Pipeline

Thuong Le Cong, Minh Kien Nguyen

September 2026

1 Introduction

Digital cameras do a surprising amount of work before they hand you a friendly JPEG. Inside the camera, the sensor measures light as a single number per pixel, often with a color filter array in front. These raw measurements are not yet a normal RGB image: they live in the camera's own color space, they may have an offset and arbitrary scale, and each pixel only knows about one color channel.

In this assignment, you will build the Camera Pipeline, the sequence of steps that transforms raw sensor data into an image that looks good on a screen. This involves cleaning noise, reconstructing colors, correcting for the monitor's color space, and adjusting for the non-linear way human eyes perceive light.

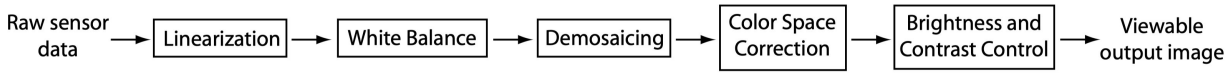


Figure 1: Overview of the Image Processing Pipeline

Programming Skeleton and Environment Setup

1. **Environment Setup**
   Before beginning, set up your environment using the pip management tool[1] (or an equivalent tool of your choice) for reliable dependency management. Run the following commands:

   ```
   python -m venv myenv
   myenv/Scripts/activate
   pip install -r requirements.txt
   ```

   Note: The provided dependencies work best with Python 3.10.11.

[1] https://viblo.asia/p/pip-pypi-package-manager-in-python-4dbZN4gL5YM

2. **Functions to Implement**
   You are provided with `assign_code.py`, which contains the `RawCameraPipeline` class. Your task is to complete the `TODO` blocks in the following functions, corresponding to the assignment sections:

   - **Sec 2:** `load_raw_tiff()` - Raw Image Extraction
   - **Sec 3:** `linearize()` - Linearization
   - **Sec 4:** `_build_bayer_masks()` - Bayer Pattern Logic
   - **Sec 5:** `white_balance(...)` - White Balancing
   - **Sec 6:** `demosaic_bilinear()` - Demosaicing
   - **Sec 7:** `color_space_correction(...)` - Color Space Correction
   - **Sec 8:** `birghtness_and_gamma()` - Brightness & Gamma
   - **Sec 9:** `save_png_jpeg(...)` - Compression

2 Raw Image Extraction

A RAW file (like .CR2) is a database of sensor data, not a ready-to-view image. It contains the raw voltage readings from the sensor pixels and metadata about how the photo was taken. To process this, we must first strip away the proprietary packaging to access the linear sensor values.

For this assignment, you will use the `campus.CR2`, a RAW image captured with a Canon DSLR[2]. You will need the dcraw command-line tool and Python libraries (`numpy`, `scipy`, `skimage`).

2.1 Reconnaissance

First, use `dcraw` to inspect the sensor's metadata, such as black levels and white balance multipliers. Run the following command:

```
dcraw -4 -d -v -w -T campus.CR2
```

Scaling with darkness $\langle black \rangle$, saturation $\langle white \rangle$, and multipliers $\langle r_{scale} \rangle$ $\langle g_{scale} \rangle$ $\langle b_{scale} \rangle$ $\langle g_{scale} \rangle$.

Report: From this line of the `dcraw` output, record the integer values of $black$, $white$, r_{scale}, g_{scale}, and b_{scale}. Include these values in your report, and use them to set `BLACK`, `WHITE`, and `WB_MULTS` in the provided Python skeleton.

2.2 Generating the Linear TIFF

Next, generate the actual image data. We want the "rawest" data possible: no color interpolation, no white balance, just the linear sensor values.

```
dcraw -4 -D -T campus.CR2
```

[2]We thank Mr. Pham Dang Hoang from the Office of Administration and Facilities Management at VNU-University of Engineering and Technology for capturing the data for this assignment.

Report: Load the TIFF file into Python using your implementation `load_raw_tiff()`. Report the bit depth (bits per pixel), image width, and image height. Convert the array to double-precision floating point and store it in `self.raw_image`.

3 Linearization

Sensors are imperfect. Even in total darkness, heat causes electrons to move, creating a "Dark Current" signal recorded as a non-zero value (the `black` level). Conversely, pixels have a maximum capacity (the `white` level). Real light intensity is proportional to the recorded value minus the black level.

You must normalize the raw data so that 0 represents absolute black and 1 represents saturation:

- Map all pixels $\leq$ <black> to 0.
- Map all pixels $\geq$ <white> to 1.
- Linearly scale the range between black and white to $[0, 1]$.

Report: Write down the specific formula you implemented to linearize the data (transforming raw value P_{raw} to P_{linear}) in your report and implement it in the function `linearize()`.

4 The Bayer Pattern

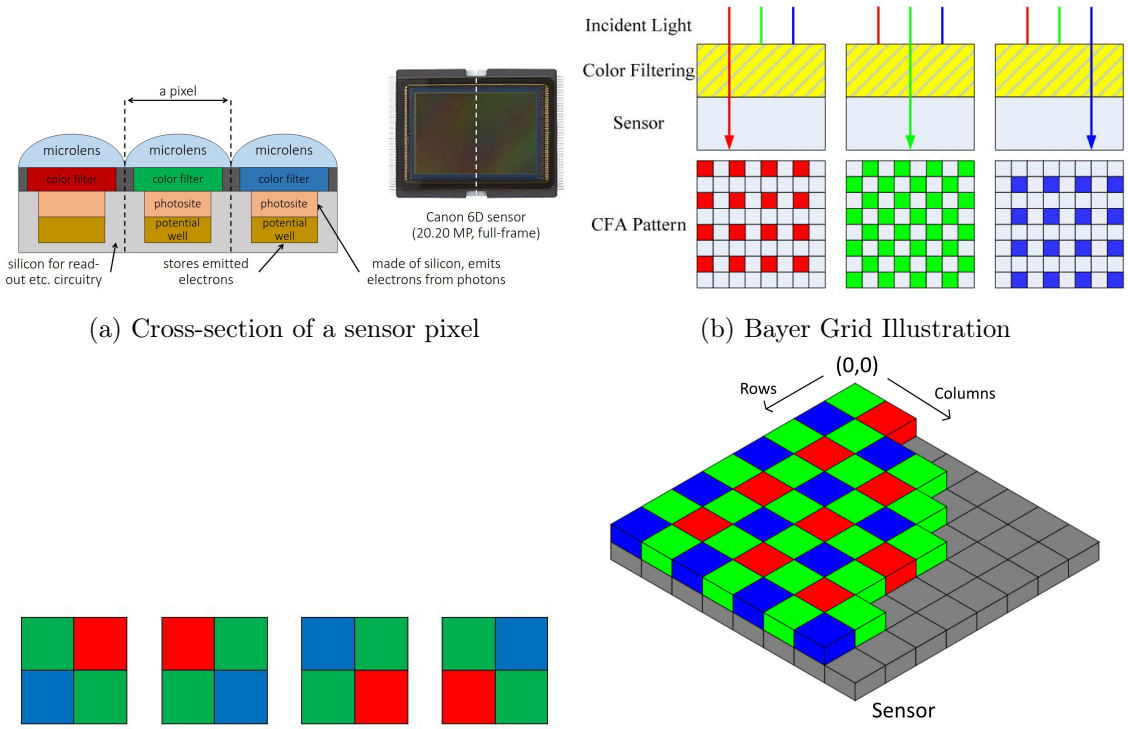
Digital sensors are inherently colorblind; the photodiodes simply count the total number of photons hitting a pixel, creating a 1-channel grayscale map of light intensity. To capture color, engineers place a specific Color Filter Array (CFA) over the sensor pixels.

Sensor Architecture As shown in Figure 2a, each pixel consists of a microlens to focus light, a specific color filter (allowing only red, green, or blue wavelengths to pass), and the photodiode which converts light into electricity. This means that at any given coordinate (x,y), the camera only records the intensity of **one** color channel.

The Bayer Mosaic There are many different CFA designs used in the industry, such as the Fujifilm X-Trans filter (which uses a 6×6 pseudo-random pattern to reduce moiré) [1]. However, the most ubiquitous arrangement is the Bayer Pattern, patented by Bryce Bayer at Eastman Kodak in 1976 [2]. It arranges filters in a repeating 2×2 grid (Figure 2b). You will notice that the pattern is not evenly distributed: there are two Green pixels for every one Red or Blue pixel.

Pattern Phase Identification While the repeating unit is always 2×2, the starting position (coordinate $(0, 0)$) determines the "Phase" of the pattern. Depending on the sensor manufacturer or how the image was cropped, the top-left pixel might be Red, Green, or Blue. This results in four possible pattern strings: RGGB, BGGR, GRBG, or GBRG, as illustrated in Figure 2c.

Report: Implement the function `_build_bayer_masks()}` in the skeleton code to generate boolean masks for the Red, Green, and Blue channels.



(a) Cross-section of a sensor pixel

(b) Bayer Grid Illustration

(c) The 4 Phase Permutations

(d) Micrograph of an actual sensor

Figure 2: The physical and logical structure of Color Filter Arrays. (a) Light passes through a microlens and filter before hitting the silicon. (b) The logical layout of the grid. (c) The four possible starting phases depending on the top-left pixel. (d) A microscopic view of a real sensor.

- Explain why the Bayer pattern is designed with 50% Green pixels compared to only 25% Red and Blue. **Hint:** *"How does this relate to the Spectral Sensitivity Function of the human eye?"*.
- Devise and report a strategy to identify the correct pattern phase for `campus.CR2` and report the correct pattern string. **Hint:** *"The correct pattern will produce natural colors, while wrong patterns will exhibit specific visual artifacts or color glitches (e.g., check the colors of trees or the sky)"*.
- Include a side-by-side screenshot comparison showing the image developed with the correct pattern versus a wrong pattern. **Note:** *"It is often easier to verify the colors if you apply white balancing before making this comparison"*.

5 White Balancing

Any object can appear to have very different colors, depending on the light illuminating it. A white piece of paper looks yellowish under candlelight and bluish under daylight, yet we still perceive it as "white" (Fig. 3). In the real world, the human visual system performs *color constancy*: it automatically discounts the illuminant so that neutral surfaces (white or gray) remain neutral under many lighting conditions.

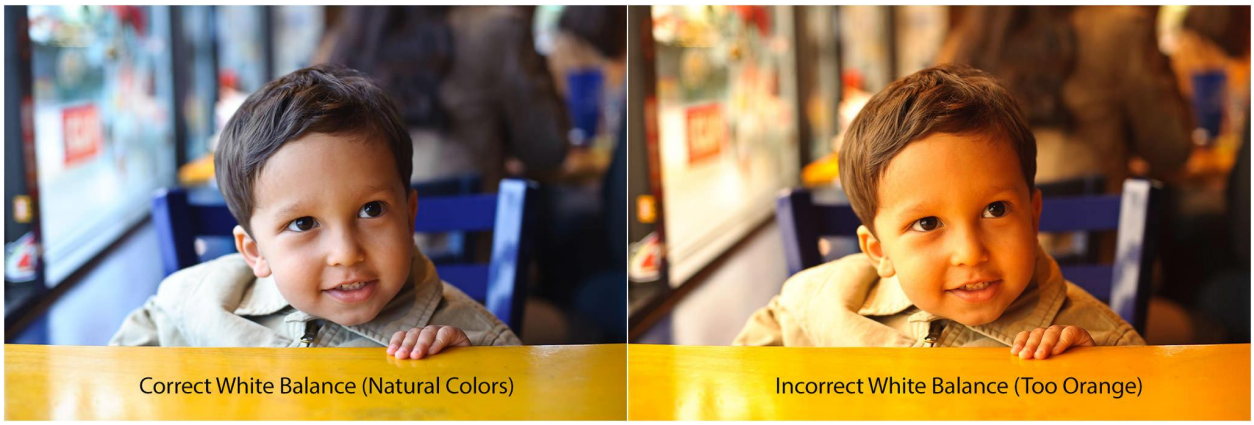


Figure 3: Illustration of white balancing

Digital cameras, however, simply measure the incoming light. If the illuminant is warm, the raw sensor values are biased toward red; if the illuminant is cool, they are biased toward blue. The sensor records the physics, not the perception. To compensate for this, the white-balancing stage in the camera pipeline applies per-channel gains to the R, G, and B channels so that surfaces that should be neutral are mapped back to neutral under a chosen reference white. In other words, white balancing is an approximate, engineering version of human color constancy: it reduces color casts from the illuminant so that the rendered image looks more natural to a human observer.

Tasks: Implement the following four algorithms to compute the scaling factors (in `white_balance()`):

1. **White World (Max-RGB):** Assume the brightest pixel in the scene is white. Scale the channels so that the maximum value of each channel equals 1 (or the maximum of the Green channel).

2. **Gray World:** Assume the average color of the entire scene is gray. Scale the channels so that the mean of the Red and Blue channels equals the mean of the Green channel.

3. **Camera Presets:** Use the multipliers you extracted from `dcraw` in Step 1. These are usually output as four numbers: $[m_!, m_2, m_3, m_4]$ corresponding to $[R, G, B, G]$.

   - Typically, the Green multipliers (m_2, m_4) are 1.0, meaning Green is the baseline.
   - If Green is 1.0, simply use $s_r = m_1$ and $s_b = m_3$.
   - If Green is not 1.0, normalize them: $s_r = m_1/m_2$ and $s_b = m_3/m_2$.

4. **Manual White Balance:** Manually select a small rectangular region (e.g., 10×10 pixels) in the image that represents a neutral object (e.g., a white wall or gray pavement). Calculate the mean values of that specific patch $(\bar{R}_{\text{patch}}, \bar{G}_{\text{patch}}, \bar{B}_{\text{patch}})$. Compute scalars that force this patch to be neutral (matching the Green channel):

$$s_r = \frac{\bar{G}_{\text{patch}}}{\bar{R}_{\text{patch}}}, \quad s_b = \frac{\bar{G}_{\text{patch}}}{\bar{B}_{\text{patch}}}$$

Report: Implement all four strategies inside `white_balance()`.

- **Visual Comparison:** Include the output images for all four methods in your report: White World, Gray World, Camera Presets, Manual White Balance.
- **Manual Details:** For the Manual White Balance results, show or describe the specific coordinate region (patch) you selected as your neutral reference.
- **Analysis:** Compare the results. Which algorithm produced the most visually pleasing (natural) result for this specific scene? Explain why you think that algorithm worked best compared to the others.

6 Demosaicing

After linearization and (optional) white balancing, we still have a sparse Bayer image. For example, the Red channel is 75% empty because Red filters only cover 1 out of every 4 pixels. We must estimate the missing values to create a full RGB image.

The simplest method is nearest-neighbor interpolation, but this results in blocky artifacts. A better standard method is Bilinear Interpolation (as in Fig. 4, Fig. 5). In this method, the value of a missing color at a specific pixel is computed as the average of its valid neighbors of that same color.

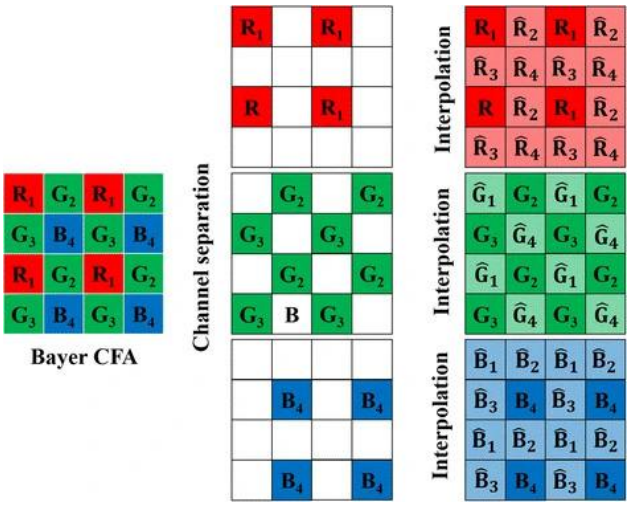


Figure 4: Illustration of the demosaicing process

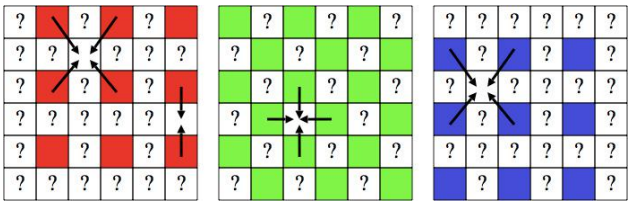


Figure 5: Demosaicing Logic

Report: Implement the function `demosaic_bilinear()` to reconstruct the full RGB image from the sparse Bayer pattern.

- You may use convolution (e.g., `scipy.ndimage.convolve`) or implement the logic from scratch. Refer to the docstring of the `demosaic_bilinear()` function for detailed hints on interpolation kernels.
- In your written report, display the resulting images for each channel (Red, Green, and Blue) separately. Verify that the channels appear dense (no black grid lines) and structurally correct.

7 Color Space Correction

The current RGB image is viewable with standard display functions, but the colors may look "weird" or washed out. This is because the pixels don't have coordinates in the correct RGB space that is expected by the operating system.

Every camera sensor has unique spectral characteristics. The specific dye used in a Canon "Red" filter is different from a Nikon "Red" filter, and both are different from the "Red" phosphor on your monitor. If we simply send the camera's RGB numbers to the screen, the colors will be wrong. This is known as a **Device-Dependent** color space. To solve this, we use a standard, **Device-Independent** color space as a bridge. The most common standard is **CIE XYZ** (defined by the CIE 1931 standard [3]), which mathematically describes how the average human eye perceives color. The transformation pipeline is:

$$\textbf{Camera RGB} \xrightarrow{\text{Sensor Matrix}} \textbf{CIE XYZ} \xrightarrow{\text{sRGB Matrix}} \textbf{sRGB}$$

We can combine these steps into a single linear matrix transformation:

$$\mathbf{I}_{sRGB} = \mathbf{M}_{Cam \to sRGB} \cdot \mathbf{I}_{Cam}$$

The sRGB standard [4] provides the following values that you can use in your implementation:

$$\mathbf{M}_{sRGB \to XYZ} = \begin{bmatrix} 0.4124564 & 0.3575761 & 0.1804375 \\ 0.2126729 & 0.7151522 & 0.0721750 \\ 0.0193339 & 0.1191920 & 0.9503041 \end{bmatrix}. \tag{1}$$

Tasks: You must construct the transformation matrix $M_{Cam \to sRGB}$ by combining two known matrices:

1. $M_{XYZ \to Cam}$: This matrix is specific to your camera sensor (Canon). It is usually found in the `adobe_coeff` function in the `dcraw` source code. For example, for a particular Canon camera, `dcraw` provides:

   $$\{6722, -635, -963, -4287, 12460, 2028, -908, 2162, 5668\}$$

   which corresponds to

   $$M_{XYZ \to Cam} = \frac{1}{10000} \begin{bmatrix} 6722 & -635 & -963 \\ -4287 & 12460 & 2028 \\ -908 & 2162 & 5668 \end{bmatrix}.$$

2. $M_{sRGB \to XYZ}$: The standard sRGB-to-XYZ matrix in (1).

Calculate the final transformation matrix:

$$M_{sRGB \to Cam} = M_{XYZ \to Cam} \cdot M_{sRGB \to XYZ} \tag{2}$$

and then invert it to get the required

$$M_{Cam \to sRGB} = (M_{sRGB \to Cam})^{-1}. \tag{3}$$

Apply $M_{Cam \to sRGB}$ to every pixel in your image inside `color_space_correction()`.

Report: Implement this computation inside `color_space_correction()`. Include the final 3×3 matrix $M_{Cam \to sRGB}$ you obtained, and show before/after color correction comparisons.

8 Brightness and Gamma Encoding

We now have a color-corrected image, but it consists of linear data directly related to photon counts. If you display this data on a monitor, it will appear significantly too dark. This is not an error in your code; it is a fundamental difference between human vision and machine sensors.

Linear vs. Perceptual. Digital sensors are **Linear**: capturing twice the number of photons results in twice the signal value. However, human vision is **Non-Linear**: we are far more sensitive to changes in dark tones than bright tones. As illustrated in Figure 6a, what a camera considers "Middle Gray" (50% intensity) looks like white to a human. True perceptual middle gray is actually only about 20% of the light intensity.

Gamma Encoding. To correct this, we apply a non-linear power curve (Gamma) to the image data before display. This curve boosts the dark values significantly while compressing the bright values.

To make the image pleasing for display, implement the following two steps (in `brightness_and_gamma()`):

1. **Brightness Scaling:** Linearly scale the image so that the mean grayscale intensity is approximately 0.25. Clip values to 1.
2. **Gamma Encoding:** Apply the standard sRGB non-linear curve:

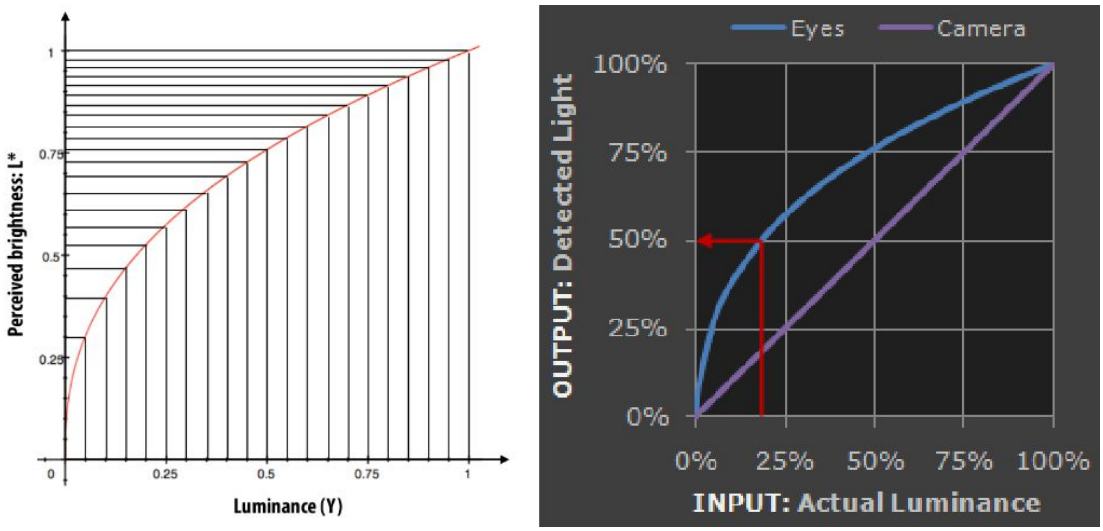
$$C_{nonlinear} = \begin{cases} 12.92 \cdot C_{\text{linear}}, & C_{\text{linear}} \leq 0.0031308 \\ 1.055 \cdot C_{\text{linear}}^{1/2.4} - 0.055, & C_{\text{linear}} > 0.0031308 \end{cases} \tag{4}$$

Report: Implement these steps in `brightness_and_gamma()`. Show a comparison between the linear image and the gamma-encoded image.

9 Compression

Save the final image as a PNG (lossless) and a JPEG (quality 95) using `save_png_jpeg()`.

Report: Calculate the compression ratio (PNG size divided by JPEG size) and find the lowest JPEG quality setting that is visually indistinguishable from the PNG. Report the chosen quality and the corresponding file sizes.



(a) Linear (Camera) vs. Perceptual (Human) (b) The Gamma Encoding Curve

Figure 6: The disconnect between sensor data and human vision (a), and the mathematical correction applied (b).

10 Instructions for Completing the Assignment

- The starter code is available on the UET Portal.
- The report template is available at `https://www.overleaf.com/read/cgdgnkbbbyjz#b9a381`.
- After completing the assignment, you need to compress all the finalized source code, resulting images, and the PDF report into a `.zip` file, name it according to the format `FullName_StudentID`, and submit it on the UET Portal.

References

[1] F. X, "X-Trans CMOS | FUJIFILM X Series & GFX - Global — fujifilm-x.com," https://www.fujifilm-x.com/global/products/x-trans-cmos/, [Accessed 06-09-2026].

[2] B. E. Bayer, "US3971065A - Color imaging array - Google Patents — patents.google.com," https://patents.google.com/patent/{U}{S}3971065{A}/en, [Accessed 06-09-2026].

[3] Commission Internationale de l'Éclairage, "Commission internationale de l'Éclairage proceedings, 1931," Cambridge, 1932. [Online]. Available: http://www.cvrl.org/database/text/cmfs/ciexyz31.htm

[4] International Electrotechnical Commission, "Iec 61966-2-1:1999. multimedia systems and equipment – colour measurement and management – part 2-1: Colour management – default rgb colour space – srgb," Standard, 1999. [Online]. Available: https://cdn.standards.iteh.ai/samples/10795/ae461684569b40bbbb2d9a22b1047f05/IEC-61966-2-1-1999-AMD1-2003.pdf